



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格



功夫到家

1920 — 2017

软件构造实践

实验六

2026春

哈尔滨工业大学（深圳）



实验目的

- 掌握Android网络开发、线程通信的实现方法
- 锻炼需求分析、技术选型、方案设计的能力



实验内容

网络功能开发

1. 联机对战，实时同步得分
2. 其他网络功能开发（选做）

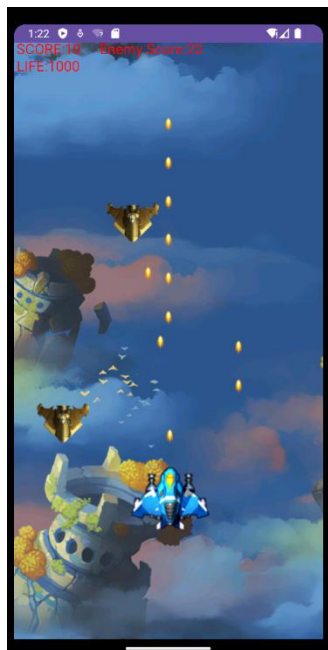


联机对战

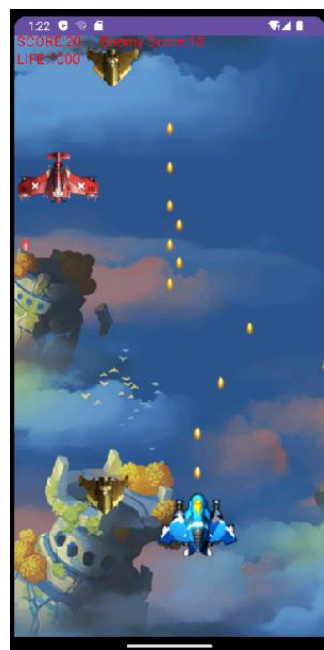
➤ 功能需求

- ❑ 在联机模式下，两名玩家可建立对战连接
- ❑ 对战结束前，任意一方分数发生变化时，另一方能实时看到更新后的分数
- ❑ 两方都死亡，则对战结束，双方页面同步展示最终分数

玩家1



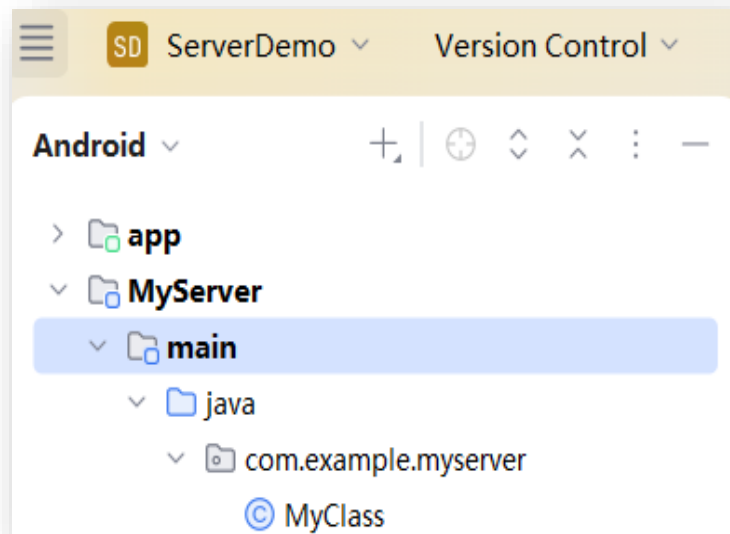
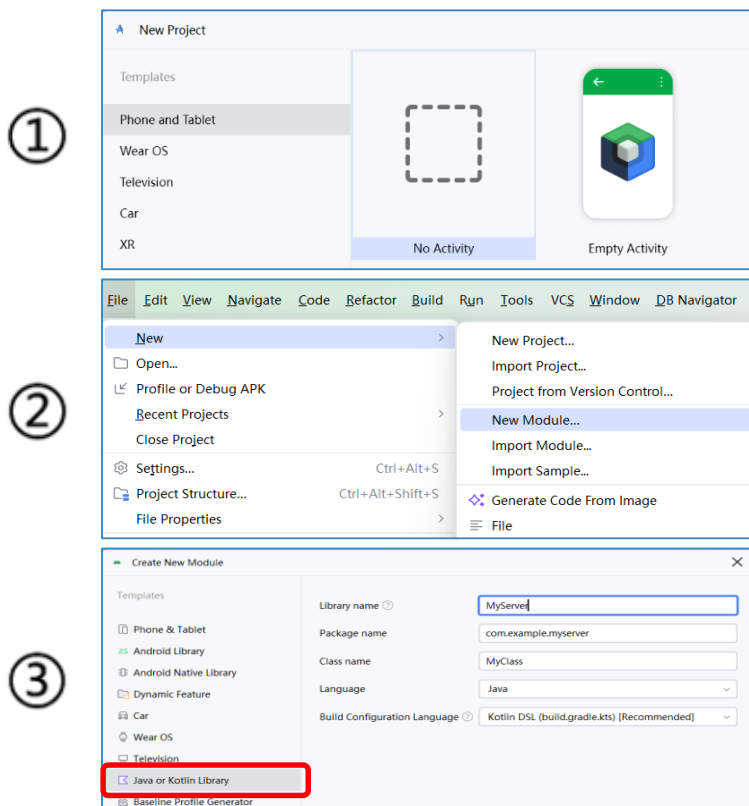
玩家2





创建Http服务器项目

- 在Android Studio中创建No Activity类型的项目
- 在项目中创建**Java Module**





创建Http服务器项目

➤ 调用HttpServer API开发服务器代码

```
public class MyClass {  
    public static void main(String[] args) throws IOException {  
        // 1. 创建服务器, 绑定8080端口, 最大并发数10  
        HttpServer server = HttpServer.create(new InetSocketAddress( port: 8080), backlog: 10);  
        // 2. 注册用户信息接口: /api/user/info  
        server.createContext( path: "/api/user/info", new UserInfoHandler());  
        // 3. 启动服务器  
        server.start();  
        System.out.println("用户信息服务器启动成功! ");  
    }  
}  
  
// 处理用户信息请求的处理器  
static class UserInfoHandler implements HttpHandler { 1 usage  
    @Override  
    public void handle(HttpExchange exchange) throws IOException {  
        // 设置响应头 (允许跨域、指定JSON格式)  
        exchange.getResponseHeaders().add("Content-Type", "application/json;charset=UTF-8");  
        exchange.getResponseHeaders().add("Access-Control-Allow-Origin", "*");  
        // 模拟根据请求参数 (userId) 返回对应信息 (此处简化为固定数据)  
        String query = exchange.getRequestURI().getQuery();  
        String responseData = USER_DATA;  
        // 若传了userId参数, 简单模拟不同用户返回  
        if (query != null && query.contains("userId=")) {  
            String userId = query.split( regex: "=")[1];  
            responseData = responseData.replace( target: "1001", userId);  
        }  
        // 发送响应 (200表示成功, 第二个参数是响应数据长度)  
        exchange.sendResponseHeaders( rCode: 200, responseData.getBytes(StandardCharsets.UTF_8));  
        OutputStream os = exchange.getResponseBody();  
        os.write(responseData.getBytes(StandardCharsets.UTF_8));  
        os.close(); // 必须关闭流, 否则客户端会阻塞  
    }  
}
```

1. 创建HttpServer实例, 绑定端口;
2. 注册请求处理器, 客户端请求特定URL时触发相应的处理逻辑;
3. 启动服务器;
4. 自定义处理器, 实现HttpHandler接口
5. 重写handle方法, 参数HttpExchange封装了请求信息和响应工具, 通过它完成和客户端的交互。



Android HTTP客户端

- 在AndroidManifest.xml中开启网络权限和明文权限
- 调用OkHttp API发起异步网络请求，注意IP和端口号的设置
- 在回调方法中处理网络响应
- 测试时先启动服务器，运行服务器main()方法，再启动Android客户端

```
public class MainActivity extends AppCompatActivity {  
    private TextView tvUserInfo; 3 usages  
    private OkHttpClient okHttpClient = new OkHttpClient(); 1 usage  
    private static final String SERVER_IP = "10.0.2.2"; 1 usage  
    private static final String SERVER_PORT = "8080"; 1 usage  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {...}  
  
    private void requestUserInfo(String userId) { 1 usage  
        String url = String.format("http://%s:%s/api/user/info?userId=%s",  
            SERVER_IP, SERVER_PORT, userId);  
  
        Request request = new Request.Builder()  
            .url(url)  
            .get()  
            .build();  
  
        // 直接在主线程调用 enqueue()  
        okHttpClient.newCall(request).enqueue(new Callback() {  
            @Override  
            public void onResponse(Call call, Response response) throws IOException {  
                if (response.isSuccessful() && response.body() != null) {...}  
            }  
  
            @Override  
            public void onFailure(Call call, IOException e) {...}  
        });  
    }  
};
```



创建Socket服务器

- 新建Android项目，创建Java Module，同创建HttpServer
- 开发Socket服务器代码：
 - ① 创建 ServerSocket，指定端口号，等待客户端连接
 - ② 客户端连接成功后，创建 Socket对象，启动子线程和客户端通信

```
public class MyClass {  
    public static void main(String args[]){  
        new MyClass();  
    }  
    public MyClass(){ 1 usage  
        try{  
            //创建server socket  
            ServerSocket serverSocket = new ServerSocket( port: 9999);  
            while(true){  
                System.out.println("waiting client connect");  
                Socket socket = serverSocket.accept();  
                System.out.println("accept client connect" + socket);  
                new Thread(new Service(socket)).start();  
            }  
        }catch (Exception ex){...}  
    }  
    class Service implements Runnable{ 1 usage  
        private Socket socket; 5 usages  
        private BufferedReader in = null; 2 usages  
        private PrintWriter pout = null; 2 usages  
        public Service(Socket socket){ 1 usage  
            this.socket = socket;  
            try{  
                in = new BufferedReader(new InputStreamReader(socket.getInputStream()));  
                pout = new PrintWriter(new BufferedWriter(  
                    new OutputStreamWriter(socket.getOutputStream(), charsetName: "utf-8")), auto  
                )  
            }catch (IOException ex){  
                ex.printStackTrace();  
            }  
        }  
        @Override  
        public void run() {  
            //处理客户端发送的信息，向客户端发送信息  
            String content = "";  
            System.out.println("wait client message " );  
            try {  
                while ((content = in.readLine()) != null) {...}  
            } catch (Exception ex) {...}  
        }  
    }  
}
```




Android Socket客户端

- 在AndroidManifest.xml中开启网络权限和明文权限
- 创建子线程，连接Socket服务器，接收数据
- 通信结束后，关闭Socket连接（客户端或服务器均可发起关闭）

```
public class MainActivity extends AppCompatActivity implements View.OnClickListener {

    @Override
    public void onClick(View view){
        if (view.getId() == R.id.btnConn) {
            new Thread(new NetConn()).start();
        }
        if (view.getId() == R.id.btnSend) {...}
        if (view.getId() == R.id.btnDiscon){...}
    }

    protected class NetConn extends Thread { 1 usage
        @Override
        public void run(){
            try{
                socket = new Socket();
                socket.connect(new InetSocketAddress
                    ( hostname: "10.0.2.2", port: 9999), timeout: 5000);
                in = new BufferedReader(new InputStreamReader(socket.getInputStream(), charsetName
                writer = new PrintWriter(new BufferedWriter(
                    new OutputStreamWriter(
                        socket.getOutputStream(), charsetName: "utf-8")), autoFlush: true);

                //接收服务器发送的数据
                String fromserver = null;
                try{
                    while((fromserver = in.readLine())!=null)
                    {...}
                    Log.i(TAG, msg: "检测到服务器关闭连接，客户端开始关闭");
                    // 客户端关闭Socket（收尾）
                    socket.close();
                    runOnUiThread() -> {...};
                }
            }
        }
    }
}
```



**同学们
请开始实验吧！**